Integration Testing I: Strategies, Reviews, and APIs

SE 413/513 Software Testing · Demo: demos/integration_testing

Dr. Kamrul Hasan, Grand Valley State University

Learning objectives

1

Explain

Explain which defects live between components, and why unit tests with mocks cannot see them.

2

Plan

Choose an integration strategy, list a system's integration points, and turn design-review findings into test scenarios.

3

Test an API

Test an HTTP API's status codes, formats, errors, and authentication, top-down with a stub below it.

Reading: Week 10, Sections 1-7. **Demo folder:** `demo-testing-suite/demos/integration_testing`

What is in the demo

LIVE DEMO 1
setup + look around

Test file	What is real	Strategy step
`test_order_service_unit.py`	only the service (all mocked)	unit baseline
`test_repository.py`	repository + SQLite	bottom-up
`test_payments_stubbed.py`	client; provider stubbed	stub at HTTP
`test_payments_live.py`	client + fake provider	bottom-up
`test_order_service.py`	service + DB + fake provider	bottom-up 2
`test_api_topdown.py`	API; service stubbed	top-down
`test_api.py`	everything	assembled

Setup

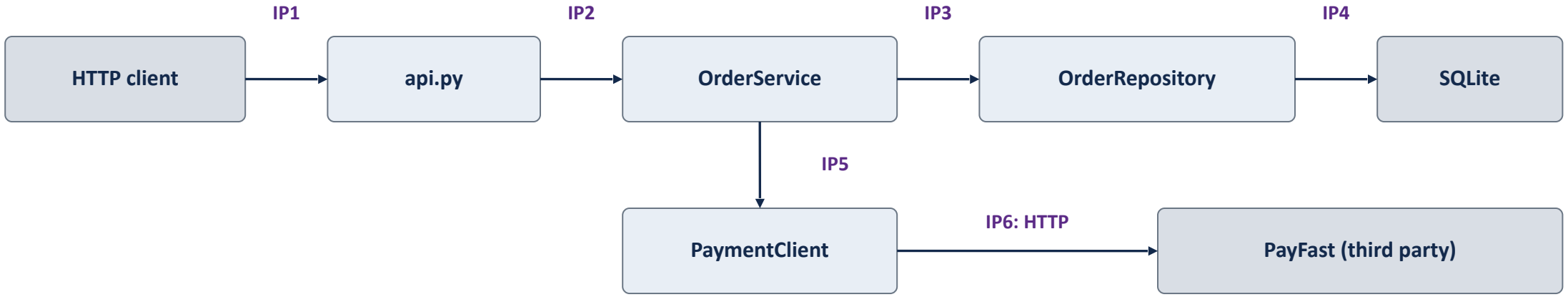
```
cd demos/integration_testing
pip install -r ../../requirements.txt
pytest
```

```
46 passed in 0.90s
```

Run from this folder

Imports, test paths, and outputs then match the slides exactly.

The shop: six integration points



IP	From → to	Over	Contract written in
IP1	HTTP client → api.py	HTTP/JSON, X-API-Key	`specs/shop_api.md`
IP2	api.py → OrderService	Python call + exceptions	`service.py, errors.py`
IP3	OrderService → OrderRepository	Python call + transactions	`repository.py`
IP4	OrderRepository → SQLite	SQL	`db.py (schema)`
IP5	OrderService → PaymentClient	Python call + exceptions	`payments.py, errors.py`
IP6	PaymentClient → PayFast	HTTP/JSON, Bearer token	`specs/payment_provider_api.md`

Good unit tests, three runs, three passes

LIVE DEMO 2
three runs, three passes

tests/test_order_service_unit.py

```
@pytest.fixture
def service(repository, payments):
    return OrderService(MagicMock(), repository, payments)


def test_declined_charge_releases_the_stock(
        service, repository, payments):
    payments.charge.side_effect = (
        PaymentDeclined("card_declined"))
    with pytest.raises(PaymentDeclined):
        service.place_order("MUG", 2, "ana@example.com")
    repository.release_stock.assert_called_once_with("MUG", 2)
```

Correct, broken client, broken service

```
$ pytest tests/test_order_service_unit.py
3 passed

$ SHOP_PAYMENTS=v1 pytest tests/test_order_service_unit.py
3 passed

$ SHOP_SERVICE=buggy pytest tests/test_order_service_unit.py
3 passed
```

Two of these runs contain a real defect. The tests are not wrong. What can a mock not have?

What a mock cannot have

1

A wire format

A mock accepts "12.50" as happily as 1250. Only the real other side knows which one its contract says.

2

Transactions

No commit, no rollback. Stock reserved for an order that was never saved is invisible to a mock.

3

Constraints

No NOT NULL, no foreign keys, no UNIQUE. The rules the database enforces never run.

Integration testing

Testing that focuses on the interactions between components or systems.

Component integration: inside one system (api.py → OrderService → OrderRepository).

System integration: between systems (the shop → PayFast, a third party).

Case study: Mars Climate Orbiter, 1999

lbf·s

produced by ground software

N·s

expected by navigation software

The spacecraft was lost arriving at Mars on September 23, 1999.

The investigation board traced the root cause to an interface: a ground software file reported thruster performance in pound-force seconds, while the navigation software that consumed it expected newton-seconds, as the project's Software Interface Specification required.

Each component did what its developers intended. The defect lived in the connection between them.

Same class of defect as this week's "12.50" vs. 1250.

Big-bang integration

LIVE DEMO 3
assert 502 == 201

Connect everything, then test the whole

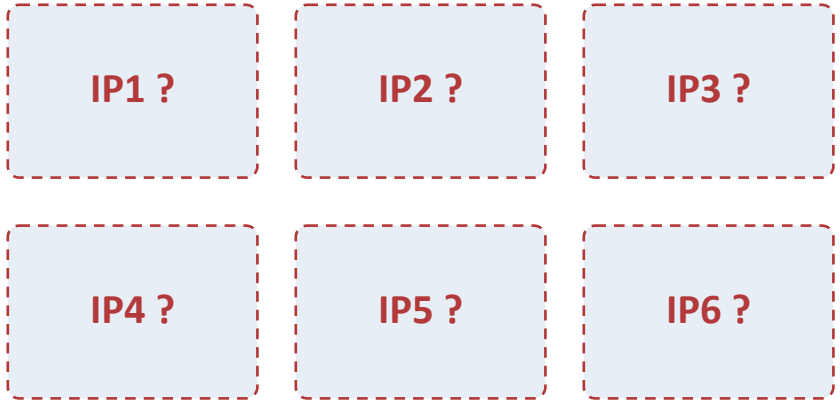
```
SHOP_PAYMENTS=v1 pytest tests/test_api.py --tb=line
```

```
tests/test_api.py:20: assert 502 == 201
tests/test_api.py:40: assert 502 == 402
FAILED test_place_an_order_and_fetch_it_back
FAILED test_declined_card_is_402_and_the_stock_comes_back
2 failed, 3 passed
```

502 Bad Gateway: the payment provider rejected something. Was it the API's validation? The total? The client's request format? The auth header? The provider?

The failing test cannot say.

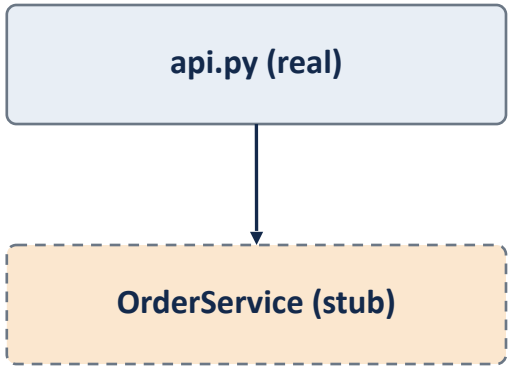
Where is the fault?



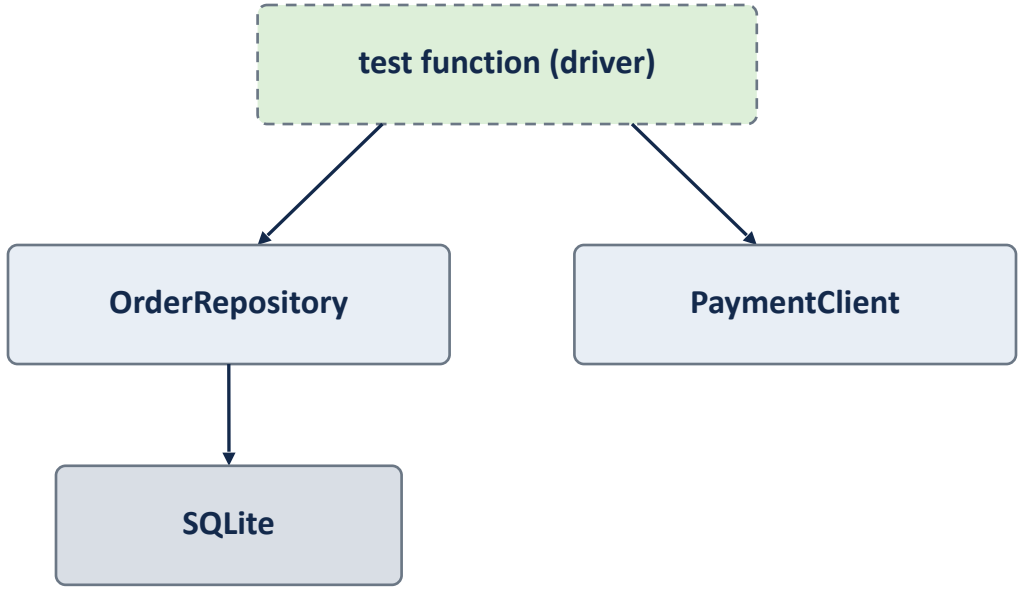
Big-bang needs no extra scaffolding, and for a tiny system it can be quickest. Its weakness is fault localization.

Incremental integration: stubs and drivers

Top-down: step 1



Bottom-up: step 1



Stub

Stands in for a component below the one being tested: it is called, and returns canned answers.

Driver

Stands in for a component above: it calls the one under test. In pytest, the test function is the driver.

Choosing a strategy

Strategy	Scaffolding	Strengths	Weaknesses
Big-bang	none	simple; fine for tiny systems	failures hard to localize; integration starts late
Top-down	stubs	early skeleton; user-visible flow tested early	many stubs; low-level risks found late
Bottom-up	drivers (tests)	risky low-level integrations tested first	whole system visible only at the end
Sandwich / risk-based	both	tests the riskiest connections first	needs planning and judgment

The demo is a sandwich. Bottom-up from the database and the payment provider, top-down from the API, meeting at OrderService. Integrate first whatever is newest, least understood, or most costly to get wrong.

Try it: map the tests

TRY IT 1
students · 5 min

1 For test_order_service.py and test_api_topdown.py: which integration points are real, which are doubled, and which strategy step is each?

```
IP1 client -> api.py          IP4 repository -> SQLite
IP2 api.py -> service         IP5 service -> PaymentClient
IP3 service -> repository     IP6 PaymentClient -> PayFast

test_order_service.py:  real = ...   step = ...
test_api_topdown.py:    real = ...   step = ...
```

2 You just changed shop/payments.py. Which test files do you run first, and why those?

```
ls tests/
# hint: which files make IP6 real, and nothing else?
```

Done early? Which integration point does no single test file isolate on its own?

Find integration problems before running anything

1

Interfaces

names, types, units, return values, exceptions

2

API specs

endpoints, formats, status codes, headers, auth, versions

3

Dependencies

availability, latency, ownership; what if it is down?

4

Data flow

does a value mean the same thing at every hop?

5

Architecture

where transactions start and end; what cannot be undone

A review is a static test: the design and the interface specifications are examined without running anything. Defects found this way are found early, when they are cheapest to fix (Fagan, 1976).

A review checklist for integration

Area	Review question
Interfaces	Do caller and callee agree on names, types, units, and which fields are optional?
Data formats	What exactly is on the wire? Numbers, dates, encodings? What does an error response look like?
Errors	Is every failure of the callee handled by the caller, and documented?
Authentication	Which credential, in which header? What happens when it is missing, wrong, or expired?
State	What must change together? What if a step fails half-way? Can every step be undone?
Dependencies	How available and how fast is it? Is there a timeout? Is a retry safe?
Data flow	Does each value keep its meaning at every hop?

Try it: review PayFast's specification

TRY IT 2
students · 8 min

1. Open the provider's published API. Walk the checklist. Write down at least four things that could make the shop and PayFast disagree.

```
less specs/payment_provider_api.md

R?  amount: ...
R?  5xx responses: ...
R?  ...
```

2. Pick one finding. Which integration point? What would go wrong? How would a test see it?

```
IP:       ...
Scenario: ...
Oracle:   ...
```

Done early? Compare with specs/design_review.md. Which findings did you miss, and why?

The review's findings

#	Finding	IP
R1	PayFast v2 requires integer cents; our client predates v2.	IP6
R2	Nothing checks that the provider receives the same total we store.	IP3, IP5, IP6
R3	Reserving stock and creating the order must be atomic.	IP3, IP4
R4	The charge cannot be inside the DB transaction. If it fails, who releases the stock?	IP3, IP5
R5	The provider's 5xx bodies are not guaranteed to be JSON.	IP6
R6	The client retries a POST that moves money. Is that safe?	IP6
R7	The first design had no timeout on the provider call.	IP6
R8	Two credentials. A wrong provider key is our fault: 502, not "retry later".	IP1, IP6
R9	Every service exception needs a documented status; Flask's default errors are HTML.	IP1, IP2
R10	SQLite ignores foreign keys unless each connection enables them.	IP4
R11	Where is input validated? Can bad input reach the service?	IP1, IP2

From review finding to test

1. **Which integration point?**
   Decides which components the test must make real.

2. **What would go wrong?**
   The scenario: the input or condition that triggers it.

3. **How would we see it?**
   The oracle: assertions on both sides of the interaction.

R4: if the charge fails, who releases the stock?

IP: IP3 + IP5. Real database (to see committed stock); fake provider.

Scenario: the provider declines after the stock was committed.

Oracle: PaymentDeclined raised; stock read through a new connection is back to 5; order status is payment_failed.

Test: `test_declined_payment_gives_the_stock_back`

Every finding ends as a test, a design change, or a documented decision. A finding that ends nowhere is a defect you have chosen to rediscover later.

What an API integration test checks

Request validation

Malformed, missing, mistyped, out-of-range fields get 400 and a clear message, and never reach the business logic.

Response validation

Documented fields, types, values; headers such as Location and Content-Type.

Status codes

Each outcome has its code: 201, 400, 401, 404, 409, 5xx.

Data formats

JSON, not HTML, including for errors; numbers as numbers; units as specified.

Error handling

Every lower-layer failure becomes a documented response, not a stack trace.

Authentication

Missing or wrong credentials get 401, and nothing else happens.

The API by hand

LIVE DEMO 4
serve.py + curl

```
python serve.py        # shop on :8000, fake PayFast on :8001

curl -i -X POST http://127.0.0.1:8000/orders -H 'X-API-Key: shop-key' \
  -H 'Content-Type: application/json' \
  -d '{"sku": "MUG", "quantity": 2, "customer_email": "ana@example.com"}'
```

```
HTTP/1.1 201 CREATED
Content-Type: application/json
Location: /orders/1

{"charge_id":"ch_1","customer_email":"ana@example.com",
 "id":1,"quantity":2,"sku":"MUG","status":"paid",
 "total_cents":2500}
```

Every error is JSON: `{"error": "missing or invalid API key"}`. A client that calls response.json() on an HTML error page crashes far from the real cause.

Now change one thing

Change	Answer
drop X-API-Key	401
`"quantity": "2"`	400
`"sku": "TEE"`	409
`--provider-mode decline`	402
`--provider-mode unavailable`	503

503 also sends Retry-After: 30

Top-down: the real API on a stub

LIVE DEMO 5
14 passed, no database

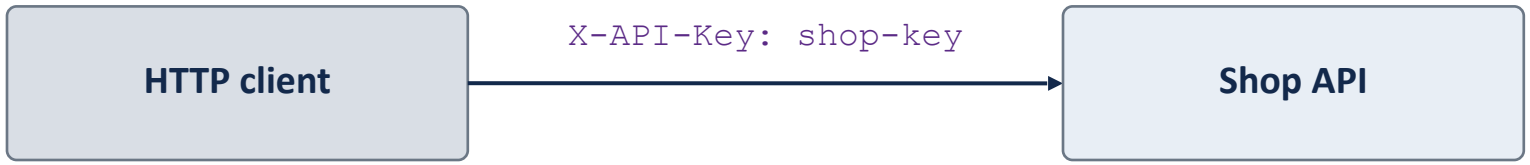
```
@pytest.mark.parametrize("raised, status", [
    pytest.param(UnknownProduct("MUG"), 404, id="unknown-sku"),
    pytest.param(OutOfStock("MUG"), 409, id="out-of-stock"),
    pytest.param(PaymentDeclined("card_declined"), 402, id="declined"),
    pytest.param(PaymentUnavailable("..."), 503, id="provider-down"),
    pytest.param(PaymentError("...: 401"), 502, id="our-bug"),
])
def test_service_errors_map_to_the_documented_status(raised, status):
    response = api(StubOrderService(raises=raised)).post(
        "/orders", json=GOOD_ORDER, headers=HEADERS)
    assert response.status_code == status
    assert response.is_json and "error" in response.get_json()
```

```
$ pytest tests/test_api_topdown.py -v
...status[unknown-sku]    PASSED
...status[out-of-stock]   PASSED
...status[declined]       PASSED
...status[provider-down]  PASSED
...status[our-bug]        PASSED
...never_reach_the_service[not-an-object] PASSED
...
14 passed in 0.12s
```

The stub records its calls: `assert stub.calls == []` proves bad input never reached the service.

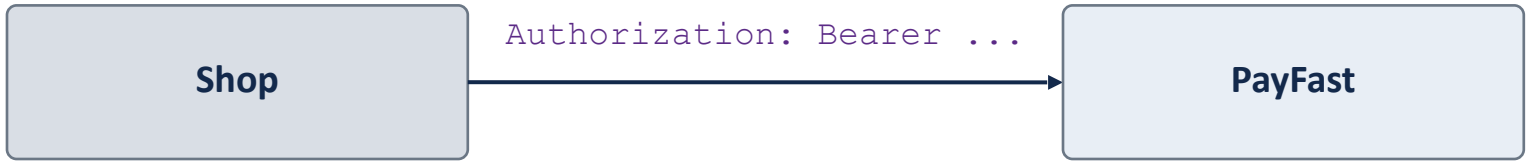
Two credentials, two different failures

Inbound (IP1)



Missing or wrong key:
401, before anything else runs

Outbound (IP6)



PayFast says 401: the shop is misconfigured.
502, no retry, not 503

Why not 503? 503 says "try again in 30 seconds". With the wrong key, every retry fails the same way. The right status is the one that tells the client what will happen next.

Try it: write two API tests

TRY IT 3
students · 10 min

1 In a new tests/test_mine.py, GET /orders/99 through the stubbed API. Assert the status and that the body is JSON.

```
from tests.test_api_topdown import (
    StubOrderService, api, HEADERS, GOOD_ORDER)

def test_unknown_order_is_json_404():
    r = api(StubOrderService()).get(
        "/orders/99", headers=HEADERS)
    assert ...
```

2 POST /orders with no body at all. What status and message? Prove the service was never called.

```
stub = StubOrderService()
r = api(stub).post("/orders", headers=HEADERS)
assert ...
assert stub.calls == []
```

Done early? Send "quantity": true. What does the API answer, and why? (Hint: Python's bool and int.)

Practice question

TRY IT
think, then vote · 2 min

With the outdated client switched in, `test_api.py` fails with `assert 502 == 201`.

Which single run points straight at the faulty integration point?

A `pytest tests/test_api_topdown.py`

B `SHOP_PAYMENTS=v1 pytest tests/test_payments_stubbed.py`

C `pytest tests/test_repository.py`

D `SHOP_PAYMENTS=v1 pytest tests/test_order_service_unit.py`

Answer: B

LIVE DEMO 6
check the answer

B `SHOP_PAYMENTS=v1 pytest tests/test_payments_stubbed.py → {'amount': '12.50'} != {'amount': 1250}`

\+ **Why** The contract check compares the request itself with PayFast's spec: integer cents, Bearer auth, Idempotency-Key. It names the field and both values.

× **A** The service is a stub: the payment client never runs.

× **C** The repository never touches payments.

× **D** Mocks pass with the bug: 3 passed.

Takeaways, and next time

1

Unit tests check the parts

Integration tests check the promises between them: formats, errors, credentials, state.

2

Integrate by risk

Incrementally, with stubs below and drivers above, so a failure points at what you just added.

3

Review, then test

Find integration risks in the design and the specs; turn each into a test, a change, or a decision.

Next time (Part 2): databases, external services, test doubles that check vs. lie, and automating integration tests.

Before then: read Sections 8-11; run python serve.py --provider-mode decline, then unavailable.